N-DIPPER: A Distributed Inter-die Peak Power Management Network for NAND Systems

Jinwoo Park, John Kim
KAIST
Daejeon, Republic of Korea
{j1018.park, jjk12}@kaist.ac.kr

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408558

***Abstract*—As NAND flash memory continues to scale, the increasing word-line (WL) stack height and higher program voltage requirements have led to severe peak-current overlaps across dies within a package. Such overlaps cause power-management-IC (PMIC) limit violations, voltage droops, and reliability degradation. Traditional static solutions – such as controller throttling or slope control – cannot fully address these issues because dynamic process, voltage, and temperature (PVT) variations and wear out-induced drift cause both nondeterministic inter-die timing mismatches and peak-current magnitude fluctuations, forcing conservative guardbands that limit system parallelism and performance. To overcome these challenges, we propose N-DIPPER (*NAND Distributed Inter-die Peak PowER Management Network*), a cooperative runtime inter-die scheduling framework that dynamically schedules high-current (HC) phases through lightweight in-package (inter-die) fabric. N-DIPPER is based on token-based scheduling across the fabric to serialize scheduling of HC phases while device-aware information is leveraged to avoid utilizing worst-case guardbands. Our evaluation results show that N-DIPPER eliminates PMIC-limit violations (100%) while sustaining 97% of the baseline throughput (without any peak power management) across diverse workloads and die configurations.**

***Index Terms*—3D NAND flash, peak current, power management, inter-die interconnect**

I. Introduction

Recent NAND flash memory systems face growing demands for high throughput and high performance [1]–[4], driven by the rapid expansion of solid-state drives (SSDs) in mobile platforms such as smartphones, tablets, and laptops, as well as large-scale cloud and AI workloads. While system-level performance relies on high I/O bandwidth and low latency, device-level NAND program latency has reached practical limits, constraining per-die throughput [5]. To meet performance requirements, modern SSDs increase system-level performance through parallelism – adding more planes per die and integrating more NAND dies per package [6].

However, greater parallelism substantially increases peak-current demand, making effective peak-current management a critical challenge [7]. When a NAND die or a plane begins a *high*-voltage operation (e.g., program or erase operations), it creates short, intense current spikes as the internal charge pumps are activated to generate the required high voltage, momentarily drawing large bursts of current [8], [9]. When multiple NAND dies or planes operate concurrently, these individual current spikes can overlap, causing the combined peak current to potentially exceed the SSD's power budget or the current limit of the power-management IC (PMIC). The aggregate peak current can have a significant impact on both reliability and performance. For example, the sudden high current demand can cause voltage droop, resulting in performance degradation of NAND charge pumps [10] and potentially lead to program/erase instability, read margin loss, or in the worst case, trigger a NAND reset [11].

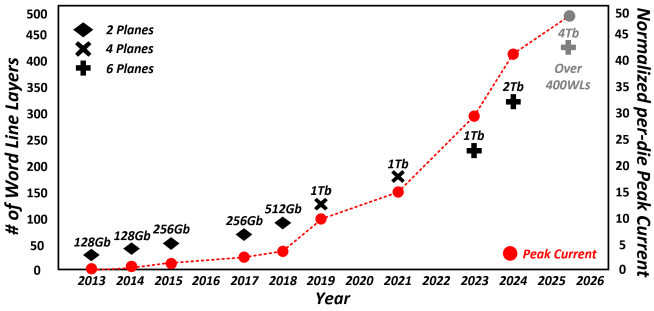


Fig. 1: Evolution of 3D NAND flash over the past decade, as the number of word lines, memory capacity, and peak current has increased.

The impact of technology on the peak current across different generations of 3D NAND over the past decade is shown in Figure 1. The figure plots the increase not only in the die memory capacity but also in the number of stacked word lines (WLs). WL stacks have grown from 24 layers in Samsung VNAND1 (2013) to 321 layers in recent SK hynix Gen9 (2024) products [12]–[16], while per-die density has reached 2 Tb featuring 4 to 6 planes. Future NAND devices are projected to exceed 400 WL layers with multi-terabit capacities [3]. The plot also shows the *estimated* per-die peak current – approximated from the product of the number of word lines and the number of planes in 3D NAND since the amount of capacitance that needs to be charged is proportional to this value. The amount of peak current has increased by over 30× during the past decade and in this work, we address the challenges of managing peak current through an inter-die interconnection network.

Modern NAND-based systems often implement a conser-

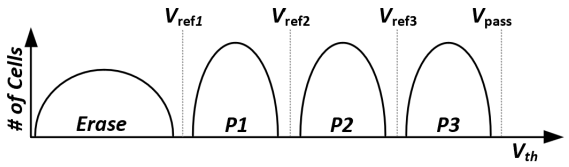


Fig. 2: NAND threshold-voltage distribution for a 2-bit MLC with three read reference voltages and pass voltage (V_{pass}).

vative peak-current management [1] where trade-off is made between performance and peak-current. For example, some approaches [17]–[19] reduce instantaneous peak current by slowly increasing the voltage while other work [20] delays NAND operations to ensure peak-current overlap does not occur; however, both approaches come at the cost of performance degradation. Recent work [21] also exploits tokens similar to this work, but uses tokens to represent current demands. In particular, it is an *on-demand* approach to manage peak currents and does not necessarily capture the PVT (Process, Voltage, Temperature) variation that can occur and results in a coarse-grained management of peak current.

In this work, we propose N–DIPPER – **N**AND **D**istributed **I**nter-die Peak Pow**ER** Management Network to avoid peak current violations with minimal performance loss. N–DIPPER is a distributed *scheduling* of peak currents (or high-current phases) across the multiple dies within a package. N–DIPPER utilizes a lightweight inter-die fabric to schedule high-current phases, while exploiting the **device-aware** per-die peak-current bounds available on each die to accurately monitor the peak current. Our analysis and evaluation show that N–DIPPER can eliminate peak-current violations while achieving 97% of the throughput on write-intensive enterprise workloads, compared to the baseline with no peak-current management. In particular, the key contributions of this work include the following.

- We identify two critical drift modes in modern 3D NAND – *Inter-die Timing Drift* and *Peak-Current Magnitude Drift* – that impact peak-currents in modern NAND.
- We propose N–DIPPER, a lightweight, in-package peak-current management architecture that eliminates peak-current violations through scheduling of high-current phases across the dies.
- We present how N–DIPPER can be scaled using multi-tokens to mitigate the fabric latency overhead while exploiting the fabric to exchange operating conditions information to more accurately represent peak currents across the dies.

II. Background

This section provides a background on NAND flash and SSD architecture, as well as an overview of NAND operations and their impact on peak currents.

[1]In this work, we use the terms peak-current management and peak-power management interchangeably, as peak power is directly proportional to the peak current under a fixed supply voltage.

A. NAND Flash Basics

Modern 3D NAND flash memory stores data in the form of a cell's threshold voltage (V_{th}), defined as the minimum gate voltage required to turn the cell on. The usable V_{th} range is partitioned into multiple discrete regions by a set of reference voltages, with each region corresponding to a logical state. For example, a 2-bit MLC cell divides the V_{th} range into four states (Erase, P1–P3) using three reference voltages (V_{ref1}, V_{ref2}, V_{ref3}) (Fig. 2), while a TLC cell further subdivides this range into eight states. During a read operation, the sensed cell voltage is compared against these reference levels to determine the stored data value. As NAND density scales from MLC to TLC, the spacing between adjacent V_{th} distributions narrows – reducing the available noise and reliability margins. Even modest shifts in V_{th} can cause a distribution to cross a reference voltage, resulting in read errors. In addition, all cell thresholds are constrained by an upper bound pass-through voltage (V_{pass}) which is applied to unselected cells during read or program operations to fully turn them on as pass transistors.

B. SSD Architecture Overview

Modern solid-state drives (SSDs) have largely replaced traditional hard disk drives in both consumer and enterprise markets due to higher bandwidth, lower latency, and improved energy efficiency. To achieve high throughput, SSDs exploit multiple levels of internal parallelism by integrating several NAND packages–each containing multiple dies and planes–and employing channel-level interleaving (Fig. 3(a)). Stable power delivery is critical for sustaining high performance [1], [22], [23]. A dedicated power-management IC (PMIC) converts the host supply (typically 3.3 V, 5 V, or 12 V) into regulated rails required by the SSD controller SoC, DRAM buffer, and NAND flash array. As shown in Fig. 3(a), the PMIC provides NAND rails such as V_{CC} (core rail for the NAND array, e.g., 2.5 V) and V_{CCQ} (I/O rail, e.g., 1.8 V), while also powering peripheral logic and DDR components. These rails must tolerate abrupt current surges from dynamic NAND operations, especially when internal charge pumps generate high WL/BL voltages during program and read phases [17]–[19], [24], [25]. A simplified 3D NAND internal architecture block diagram that illustrates WL biasing in program operations is shown in Fig. 3(b). At the package level, multiple dies share a common power rail, and the PMIC typically enforces a strict peak current limit constrained by the system's power budget. If the sum of the current across all of the dies exceeds this budget, voltage droop on the shared rail degrades timing margins, slows charge-pump transitions [26], [27], and may trigger brownout [2] or reset behavior via voltage-monitor circuits [11], potentially aborting ongoing operations.

C. 3D NAND Operations

The key operations in a 3D NAND include program (or write) operation, read operation, and erase operation. During

[2]Brownout is a supply-voltage droop below a protection threshold that can trigger voltage-monitor circuits to reset or abort ongoing operations.

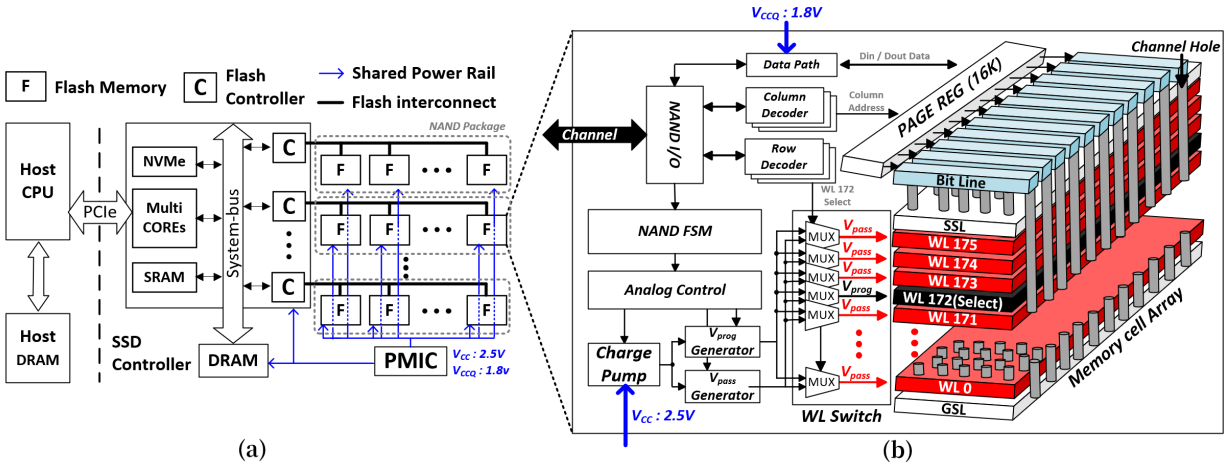
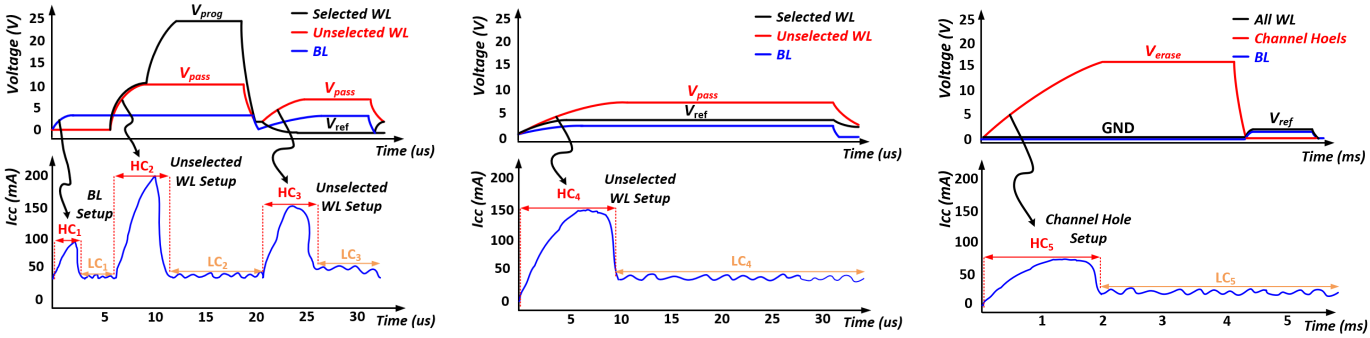


Fig. 3: (a) High-level block diagram of a modern SSD system including the PMIC (Power Management Integrated Circuit), and (b) detailed 3D NAND block diagram illustrating word-line (WL) biasing for program operations.



(a) Program loop operation. (b) Read operation. (c) Erase operation.

Fig. 4: Illustration of word-line (WL) and bit-line (BL) voltages setup phases and the corresponding high-current peaks or instantaneous current (I_{CC}) for (a) program, (b) read, and (c) erase operations. The program loop operation represents a single program pulse followed by a verify operation. High-current (HC_i) and low-current (LC_i) phases are annotated on the diagrams.

these operations, the internal voltage generators raise the WL and BL to a target voltage level. Charging these large capacitive structures results in high-current (HC) phases, whereas quasi-static bias-hold or sensing phases appear as low-current (LC) phases. Example waveforms for the three major NAND operations are illustrated in Fig. 4. Each operation consists of one or more high current (HC) phases where precharge draws high peak current, followed by a low-current (LC) phase when relatively small, steady currents are drawn.

- **Program Operation (Fig. 4a):** The first high current phase (HC_1) occurs during BL setup (or bit-line precharge) followed by LC_1. The next high-current phase (HC_2) occurs during WL setup for both select and unselected WLs. While V_{pass} for the unselected WL is lower than V_{prog} ($V_{\mathrm{pass}} \approx 10$ V [17], [18], [24]), charging the large number of unselected WL layers generates high peak currents from the large amount of aggregate capacitance. The last stage is to verify whether target V_{th} has been reached and requires the unselected WLs to be precharged.
- **Read Operation (Fig. 4b):** The read operation is similar to the verify stage during the program operation as precharging of the many *unselected* WLs to V_{pass} (HC_4) is followed by low-current sensing (LC_4). The read operation is often longer than the verify stage as multiple reference comparisons are needed to reliably decode the cell's logical state.
- **Erase Operation (Fig. 4c):** Erase resets a block of NAND cells by channel hole setup (HC_5) where a high channel voltage V_{erase} (15–20 V) is applied to the channel (or substrate) while all WLs are grounded. Unlike other operations, erase holds WL at a constant bias and avoids simultaneous WL voltage charging – making its peak current substantially less sensitive to the number of WL layers.

III. Motivation

Peak-current violations represent a critical issue in high-density NAND systems as they not only degrade data integrity but also jeopardize the forward progress of program

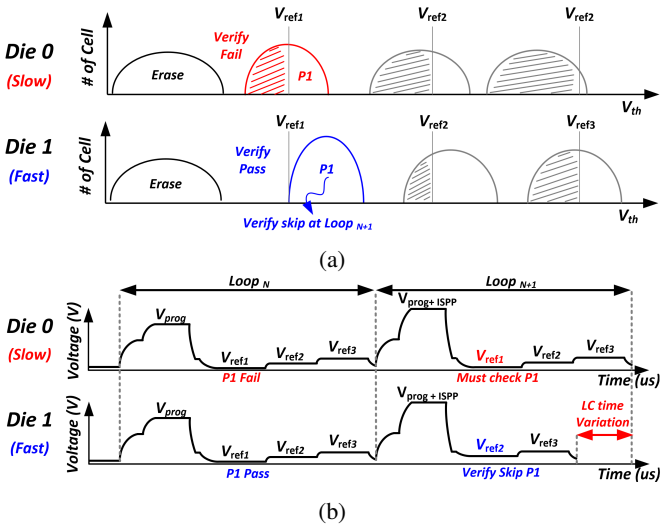


Fig. 5: (a) Different threshold voltage trajectories between a slow and a fast die and (b) the resulting verify-time variation.

operations. As a result, peak-current management is necessary for performance, reliability, and long-term device health [1], [23]. Peak currents that can occur during NAND operations (in particular the program operation) can be problematic when the peak-current or HC phases *overlap* between the dies. In this section, we highlight the challenges of peak current management and analyze why existing approaches are limited in managing the inter-die timing drift that can occur from PVT and wear state, especially as die-level parallelism scales.

A. Inter-die Timing (Horizontal) Drift

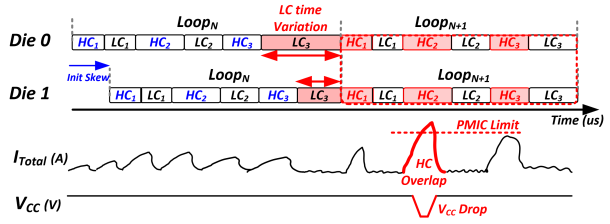
Timing drift changes *when* HC occurs (horizontal shift in time), as internal loop/verify behavior diverges across dies during `Busy`. Static peak-current management [17]–[20] rely on fixed assumptions about the timing and current behavior of NAND operations to avoid peak-current violations under worst-case conditions. As a result, static management must adopt conservative throttling or assumption – often sacrificing performance and underutilizing available parallelism. One of the challenges is that the SSD controller has limited visibility into the internal timing within each NAND die. When the SSD controller issues a NAND command (e.g., program/read/erase), the device enters a `Busy` state and performs a sequence of internal analog and verify steps. While `Busy`, the NAND does not accept new commands and exposes no intermediate status to the controller – thus, the only observable events are the beginning and completion of `Busy` state. In particular, NAND employs incremental step pulse programming (ISPP), where a program operation consists of multiple program-verify loops that repeat until cells' threshold voltage meets the target window [25], [28] to achieve a tightly controlled threshold-voltage (V_{th}) distribution. Since loop counts depend on cell-level variation and target states, the internal execution time can vary across dies and across operations. This variation accumulates over loops and contributes to unpredictable timing of when high-current phases occur and overlap – which we refer to as *inter-die timing drift*.

Thus, even if all dies begin program operation simultaneously, their threshold-voltage (V_{th}) trajectories, or the rate at which cells approach their target threshold levels, diverge due to process variation, temperature, and wear [29]–[31]. For example, a *slow* die keeps more cells in lower states (e.g., P1) and requires additional verify pulses, while a *fast* die stops verifying lower states earlier and proceeds to higher states sooner (Fig. 5(a)). This divergence in programming speed directly results in verify-time variation between the dies, particularly in the LC_3 phase [29]. Since modern NAND controllers adopt selective-verify policies to shorten t_{PROG}, once a die passes P1, the P1 verify step is skipped in subsequent loops [29], [32]. As a result, the slow die performs verify steps at P1→P2→P3, while the fast die skips P1 and verifies only P2 and P3 during $\mathrm{Loop}_{(N+1)}$ (Fig. 5(b)). This verify-time variation (or the LC phase time imbalance) can accumulate across loop iterations and create uncertainty on when each die enters into HC phases.

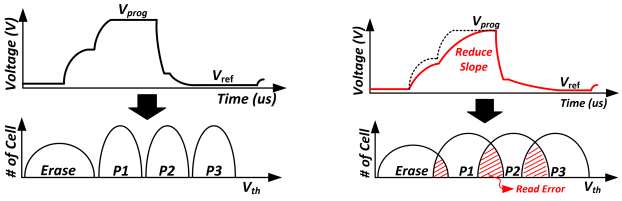
As a result, even though the HC phases may be initially skewed (Fig. 6(a)), the verify-time variation during the *LC* phases results in an inter-die timing drift such that the HC phases eventually overlap and potentially introduce peak-current violations beyond the PMIC limit [1], [23]. Exceeding the PMIC limit induces a V_{CC} droop on the shared supply rail [10], [23] which compromises the driving capability of internal charge pumps, reducing the rising slope of V_{prog} and V_{pass} [17]–[19], [26], [27]. As shown in Fig. 6(b), the reduced slope prevents WLs from settling to their target levels in time – widening the threshold-voltage (V_{th}) distributions and increasing bit errors during state transitions [23], [30], [33]. Under severe overlap, V_{CC} may fall below the internal reference V_{REF}, triggering an on-die brownout/reset event that can abort the ongoing program operation [11]. Thus, *dynamic* peak-current management across NAND dies is necessary to ensure high performance while addressing the challenges of inter-die timing drift.

B. Peak-Current Magnitude (Vertical) Drift

While inter-die timing drift changes *when* HC phases occur (a horizontal shift in time), *magnitude drift* changes *how much* current an HC phase draws (a vertical shift in current). Another challenge with static peak-current management is that peak-current in NAND will vary over time from wear and temperature [34]–[36], which we refer to *peak-current magnitude drift*. The key intuition is that a NAND die does not consume a fixed amount of current for the same operation. Manufacturing variation shifts the die's peak, and even within one die, the peak can vary with the target wordline/layer because different wordlines present different capacitive loads to be charged [9], [17]. An example illustration of how peak current per-die can change over time is shown in Fig. 7(a) – for example, initial per-die, peak current can be 200mA but can increase to up to 220mA [34], [37]. To address the potential peak-current magnitude drift, static approaches need

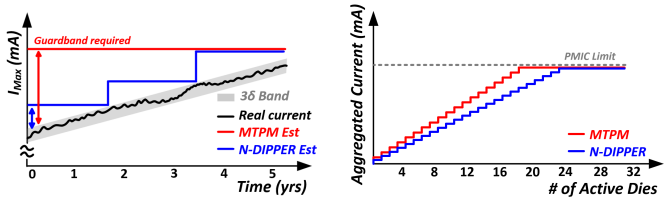


(a) Package-level V_{CC} droop caused by HC overlap.



(b) Stable operation (c) Degraded operation

Fig. 6: (a) Inter-die timing drift that results in high-current (HC) overlap between two dies; (b) stable and (c) degraded operation from V_{CC} droop and the reduced voltage slope.



(a) (b)

Fig. 7: Peak-current magnitude drift and its impact on static management. (a) Per-die peak current drifts over time. (b) Aggregated current versus the number of active dies, showing earlier PMIC-limit reach under coarse tokenization (MTPM) than under N-DIPPER.

to provide a conservative estimate or a significant guardband (i.e., overhead) to ensure that peak current violation does not occur.

Moreover, the conservatism of static management can be amplified by how peak current is *represented* and *budgeted* in practice. For example, prior work [21] has also proposed to leverage tokens to manage peak-power in NAND-based SSD systems. However, the usage of tokens differs as their tokens represent the amount of peak current that can be leveraged and thus, multiple tokens need to be grabbed when entering high-current phases and then, subsequently released when completed. When current is tokenized, the required tokens are computed via coarse quantization (often with rounding-up), so each HC authorization can over-approximate its true peak by up to one token unit. As more dies run concurrently, this rounding-up error can trigger premature throttling even when the true summed current fits within the package budget. In addition, such static approach can limit the scalability of the NAND system, especially as the number of dies increases. By assuming a conservative limit on the peak current, the number

eFuse block	Field (symbol)	#Ent.	b/ent.	Encoding / notes
HC Base eFuse	Peak magnitude	5	8	1 mA / LSB
	Peak duration	5	8	80 ns / LSB
P/T/W Offset eFuse	Offset (ΔI_{PTW})	135	8	$(5\times3\times3\times3)$ 8-bit signed 1 mA / LSB

TABLE I: Per-die eFuse tables used to derive the effective HC peak-current bound.

Device State	State Parameter	Description	Source of Information
Process Corner (P)	SS	slow	Static eFuse
	TT	typical	
	FF	fast	
Temperature (T)	CT	Cold Temp	On-die Temp Sensor
	RT	Room Temp	
	HT	Hot Temp	
Wear State (W)	Init	Initial	Flash Controller (P/E Count)
	Mid	Moderate	
	Worn	Aged	

TABLE II: Device states and parameters used in N–DIPPER to select the peak current offset added to the nominal peak current.

of parallel, active dies is limited – e.g., when scaling to 32 dies, static approaches can limit the number of parallel dies to approximately 18 dies while dynamic approach proposed in this work enables better scalability as up to 24 dies can be active in parallel. In practice, a static scheme must reserve a conservative peak-current allowance for each die (including guardbands). Under a fixed package current budget, larger per-die allowances directly reduce how many dies can be active at the same time. Magnitude drift increases the required guardband, and token-based estimation can further inflate the assumed per-die peak due to rounding-up. As a result, the maximum safe parallelism shrinks as die-level parallelism scales (Fig. 7(b)).

IV. N-DIPPER Architecture

In this section, we present N–DIPPER – NAND Distributed Inter-die Peak PowER Management Network – which exploits a die-to-die interconnection network for peak-current management. In particular, we present the design of the network and microarchitectural changes needed within each die to support dynamic peak-current management.

A. N–DIPPER Overview

N–DIPPER leverages the die-to-die network to exchange information on when high-current (HC) phases are entered, ensuring that HC phases across dies are scheduled to avoid peak-current violations. In particular, token-based scheduling is leveraged to address inter-die timing drift (Sec. III-A) while device-aware management minimizes the impact of peak-current magnitude drift (Sec. III-B). The key components of N–DIPPER include the following.

- **Interconnection Network:** Modern NAND dies (and package) do not have die-to-die direct communication

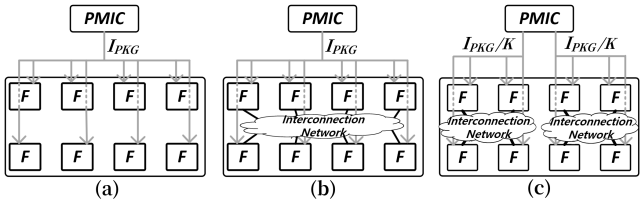


Fig. 8: NAND package-level inter-die connectivity for (a) baseline with a shared power rail, (b) N–DIPPER where dies are connected via lightweight interconnection network, and (c) multi-token N–DIPPER with K subgroups, each subgroup having a peak-current limit of I_{PKG}/K.

Fig. 9: Packet format in N–DIPPER for (a) token packet and (b) metadata packet. One bit header differentiates the packet type.

(Fig. 8(a)) but N–DIPPER introduces a lightweight, in-package die-to-die fabric for peak-current management (Fig. 8(b)). The die-to-die fabric enables a *distributed* peak-current management that is *independent* of the SSD controller. For scalability, N–DIPPER supports a multi-token extension that partitions the package into K subgroups, each with an independent fabric and an allocated peak-current budget of I_{PKG}/K (Fig. 8(c)).

- **Token-based Scheduling:** A single, global token is circulated across the fabric to schedule when each die can enter high-current (HC) phases while ensuring peak-current violation does not occur. Each die maintains the total peak current that can be drawn (I_{sum}) at a given time and it is compared against the PMIC limit (I_{pkg}). In addition, instead of being an on-demand management of peak current, N–DIPPER "schedules" high-current phases. Thus, the token is not held during HC and concurrent execution of HC across multiple dies is possible since safe overlaps are scheduled.
- **Device-Aware Peak-Current Management:** To enable device-aware estimate of peak current magnitude drift, N–DIPPER exploits eFuses [3] – to store per-chip parameters that N–DIPPER leverages to compensate for manufacturing variation and operating conditions. Per-die information (or metadata) is exchanged through the die-to-die fabric when scheduling HC phases and a local look-up performed based on the metadata to estimate the peak current.

B. Microarchitectural Components

A high-level block diagram of N–DIPPER is shown in Fig. 10. The key components are described below.

[3]Electrical fuse or eFuses are one-time programmable hardware elements that are located on-die in the peripheral region and behave like a tiny read-only memory at runtime.

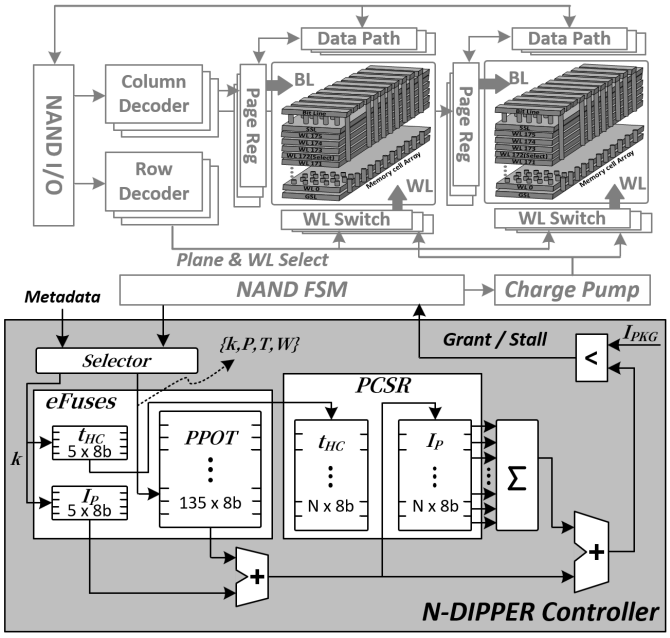


Fig. 10: High-level block diagram of N–DIPPER controller within each NAND die.

1) Die-to-Die Fabric: N–DIPPER introduces a lightweight inter-die fabric to interconnect the dies together. However, unlike general-purpose interconnection networks [38], [39], the constraints of NAND packaging present strict constraints on the network architecture and topology – including limited I/O ports and limited area to introduce any router and/or buffer. As a result, N–DIPPER utilizes a very low-radix topology (e.g., unidirectional ring) and minimizes logic overhead by reusing the NAND datapath while only supporting two types of packets – token packets and metadata packets Fig. 9. Traditional "flow control" is not necessary as there is only a single packet in-flight in N–DIPPER as the token is circulated to determine which die can enter the high-current (HC) phase and only the die with the token is allowed to inject the metadata packet.

2) Token: A single token is circulated among the dies and is grabbed by the dies to determine whether the current die can enter high-current (HC) phase or not. A token packet is sent only when the token is handed over. Therefore, TK=0 is never transmitted (token packets always carry TK=1 when sent). By having only a single token, N–DIPPER ensures that only a single die can enter HC phase at a given time and prevents the package peak current from being exceeded.

3) Metadata: Once a token is obtained and HC entry is granted, a metadata packet is broadcast to all other dies, which ***provides information about HC phases that has been currently scheduled.*** Instead of transmitting the full peak current offset values (which would require 9 bits to represent a $\sim$300 mA-level current at 1 mA resolution), discrete information about the current HC phases are transmitted which allows remote dies to understand the characteristics of the current

HC phases and locally calculate the impact of the current HC on the peak current. In particular, the information sent to the other dies includes the following – source ID of the die which had HC scheduled, the type of HC phase being scheduled, and additional information about the current die, including the processor corner (P), temperature (T), and the wear state (W) (Fig. 9). The potential values for the different operating conditions are summarized in Table II. `P` is readily available from existing eFuses on NAND dies while `T` is determined based on the temperature sensors [40], [41]. In comparison, `W` is based on the P/E cycles and is provided by the SSD controller at coarse timescales [42].

4) eFuses/Registers: Electrical fuses (eFuses) in NAND flash are one-time programmable, non-volatile bits used to store device-specific configuration and characterization information determined during manufacturing or qualification and not modified [8]. For N–DIPPER, we leverage existing eFuses to store the HC duration, while proposing additional eFuse fields to (i) store a *nominal* peak-current bound and (ii) compensate for peak-current drift under different operating conditions. Two sets of eFuses ($\texttt{eFuse.I}_p\texttt{[k]}, \texttt{eFuse.t}_{HC}\texttt{[k]}$) are used where $\texttt{I}_p\texttt{[k]}$ represents the statistically determined 3σ upper bound of the baseline peak current, $\texttt{t}_{HC}\texttt{[k]}$ is the amount of time for the high-current (HC) phase during NAND operation, and k represents the different HC phases(Fig. 4). [4] In addition, the following register and eFuses are also needed for N–DIPPER.

- **Per-Die Peak Current Status Registers (PCSR):** Each die locally maintains the status of the other $N-1$ dies in `PCSR` with two values – including their peak current usage (I_p) and the remaining time of their HC usage (t_{HC}). $\texttt{PCSR[i].t}_{HC}$ is used to determine if die i is utilizing a peak current amount of $\texttt{PCSR[i].I}_p$.
- **PVT-Wear state Peak-Current Offset Table (PPOT):** To compensate for the peak-current magnitude drift [34], [35], a lookup table (populated with signed peak-current offsets) is introduced, which is indexed by the die's operating-condition selectors, including the HC phase ID and the shared metadata selectors.

`PCSR` is maintained as a register since it needs to be constantly modified while `PPOT` is written once and not modified – thus, `PPOT` can be programmed using eFuses[5]

C. N–DIPPER *Controller*

An algorithm describing N–DIPPER controller is presented in Algo 1 which consists of three segments. The first segment is processing token packets – when the token is received (and there is a request for HC entry), the local die determines whether HC can be entered without exceeding the peak current budget. The effective peak current is calculated by summing up the neighbors' peak-current usage (Line 5) and then, adding the nominal peak current usage for the current $\texttt{HC}_k$, as well as the current offset based on current die operating conditions and using the `PPOT` lookup table. If the current effective peak current exceeds the packaging limit (I_{pkg}) (Line 6), the die holds on to the token and stalls until peak current limit is no longer violated – i.e., when other dies finish their HC phases. Once the die can enter the HC phase, it broadcasts its metadata to the other dies (to ensure that other dies are aware of the current die's HC entry) and effectively "synchronize" all of the dies on the peak current usage. Once metadata broadcast is completed, the HC phase can begin and the token is transmitted to the neighboring die (Line 12). In addition, if there is no request for HC phase in the current die, the received token is also immediately forwarded to the neighboring node as well (Line 12).

The second segment consists of processing the metadata that is received. The information received from the neighboring dies (Line 15) is used to update the `PCSR` register based on the nominal values of the peak current (and duration) information for $\texttt{HC}_k$ as well as the peak current offset (Line 16- 17). Afterwards, the metadata is forwarded to the other dies. The last segment of the controller is updating the `PCSR`. Since `PCSR` stores how long the HC phase will last in die i, values are decremented each cycle until it reaches a value of zero

Algorithm 1 N–DIPPER controller Overview.

```
    NDIPPER_CONTROLLER() {
 1: for each cycle do
        // Stage1: Process token packet
 2:     if received_token then
        // k: local HC phase
 3:         if ∃ HC_request(k) then
 4:             sel ← {k, P, T, W}                          ▷ Read local state
 5:             I_p_sum ← Σ_{i=0}^{N-1} PCSR[i].I_p
 6:             while I_p_sum + eFuse.I_p[k] + PPOT[sel] > I_pkg do
 7:                 Wait()          ▷ stall until peak current budget available
 8:             end while
 9:             Broadcast(metadata)                          ▷ blocking operation
10:             HC_start()                                ▷ non-blocking operation
11:         end if
12:         Send_token()              ▷ forward token to neighboring node
13:     end if
        // Stage2: Process metadata packet
14:     if received_metadata then
15:         {src, k, sel} ← metadata[]
16:         PCSR[src].I_p ← eFuse.I_p[k] + PPOT[sel]
17:         PCSR[src].t_HC ← eFuse.t_HC[k]
18:         Forward(metadata)
19:     end if
        // Stage3: Update local status register (PCSR)
20:     for i = 0 to N − 1 do
21:         if PCSR[i].t_HC > 0 then
22:             PCSR[i].t_HC−−
23:             if PCSR[i].t_HC == 0 then
24:                 PCSR[i].I_p ← 0          ▷ reset peak current contribution
25:             end if
26:         end if
27:     end for
28: end for
    }
```

[4]We assume $\texttt{I}_p\texttt{[k]}$ is a die-common eFuse field programmed at wafer final test. For each HC type k, final-test characterization profiles the peak-current distribution under nominal conditions and programs a conservative bound (e.g., $\mu + 3\sigma$) identically for all dies. The corresponding HC duration $\texttt{t}_{HC}\texttt{[k]}$ is characterized in the same flow and stored as well.

[5]The hardware overhead of eFuses is lower than registers because eFuses are non-volatile, read-mostly storage programmed once (at test/qualification), eliminating the clocked, write-capable storage and update circuitry required by registers.

TABLE III: **Simulation parameters for each** N–DIPPER **and flash-memory configuration.**

Category	Parameters
NAND Device-Level Simulator	HC Max = 200 mA, PMIC = 1.2A (8 dies)/2.4A (16 dies)/4.8A (32 dies)
SSD Platform (SimpleSSD)	PCIe 4.0 ×4, system-bus = 8 GB/s, 8 channels, 4 ways, total die count = 8 / 16 / 32
N–DIPPER Topology	Ring (unidirectional / bidirectional), Mesh
Flash Memory Timing	Read = 50 μs, Program = 426 μs, Erase = 5 ms

which represents the time when the HC phase is completed and thus, the current contribution to overall package peak current can be reset to 0.

D. Multi-Token Extension

As the number of dies (and thus the network hop count) increase, the interconnect overhead latency (including the token latency as well as the metadata broadcast latency) proportionally increases as well. To mitigate this overhead, N–DIPPER can be extended with *multi*-token where the package is partitioned into K subgroups, and each subgroup maintains an independent token (Fig. 8(c)). This creates K concurrent physical inter-die fabrics and reduces the effective hop count by a factor of K. Note that multi-token N–DIPPER does not increase the available package current budget, as I_{pkg} remains fixed; however, N–DIPPER allocates a fraction of I_{pkg}/K to each subgroup and enforces it ensure overall peak current violation does not occur. This introduces a trade-off as larger K reduces the inter-die fabric latency overhead but decreases the amount of peak current to each sub-group – thus, potentially preventing HC overlap to cause performance degradation. In addition, if the peak current usage across the dies are not balanced, performance can degrade since peak-current will be managed "conservatively" from the static partitioning of the peak current across the sub-groups.

V. Evaluation

A. Methodology

To evaluate the proposed N–DIPPER architecture, we used a custom NAND device-level simulator as well as an architectural simulator (SimpleSSD [43]). The custom simulator was used to model realistic NAND flash runtime dynamics to understand the impact of peak-current management. The simulator introduced random variations in the program loop counts and verify-time durations across each NAND program operation to model a commercial NAND behavior, based on SK Hynix's 176-layer TLC NAND [15] with tPROG of 425 μs. We also model read and erase current profiles to validate that N–DIPPER's scheduling logic correctly regulates peak current under mixed-operation traffic. The SimpleSSD simulator [43] was used to evaluate the potential degradation of peak-current management on overall system performance by adding scheduling overheads from the device-level simulator to per-operation NAND latencies (e.g., t_{PROG}). The configuration used in the SSD simulation is summarized in Table III.

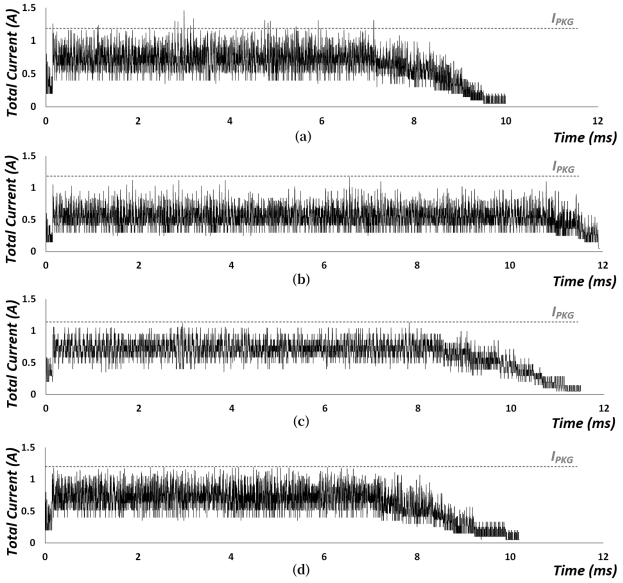


Fig. 11: Total peak current for an 8-die package with 1.2 A PMIC budget under a synthetic sequential write workload using (a) Baseline (with no peak-power management), (b) SCT, (c) DWS, and (d) N–DIPPER.

We simulate packages with 8, 16, and 32 dies sharing a power rail. The PMIC budget (I_{pkg}) scales linearly with the die count (1.2 A, 2.4 A, and 4.8 A, respectively) based on a nominal HC peak of 200 mA. Workloads include synthetic sequential write streams with varying Read/Write ratios (R100 to R0 where R100 is 100% read accesses) and real-world traces (Web1, TPC-C, MSR1) from SNIA [44].We compare the following peak-current management policies:

- **Baseline:** No peak-current management (thus can violate the package-level current budget) but provides an upper bound on performance.
- **SSD Controller Throttling (SCT)** [20], [45]: Controller-level scheduling that caps peak current by limiting concurrent operations across dies/planes.
- **Device Waveform Shaping (DWS)** [10], [19]: Device-level charge-pump slew-rate throttling that flattens HC peaks by slowing internal voltage ramps.
- N–DIPPER (this work): Different physical topology, including ring and mesh are evaluated, as well as single and multi-token implementations.

B. Peak Current Reduction

The impact of different peak-current management is shown in Fig. 11 for an 8-die NAND package with a 1.2 A peak-current budget (I_{pkg}), when running a synthetic sequential write workload designed to saturate inter-die parallelism,

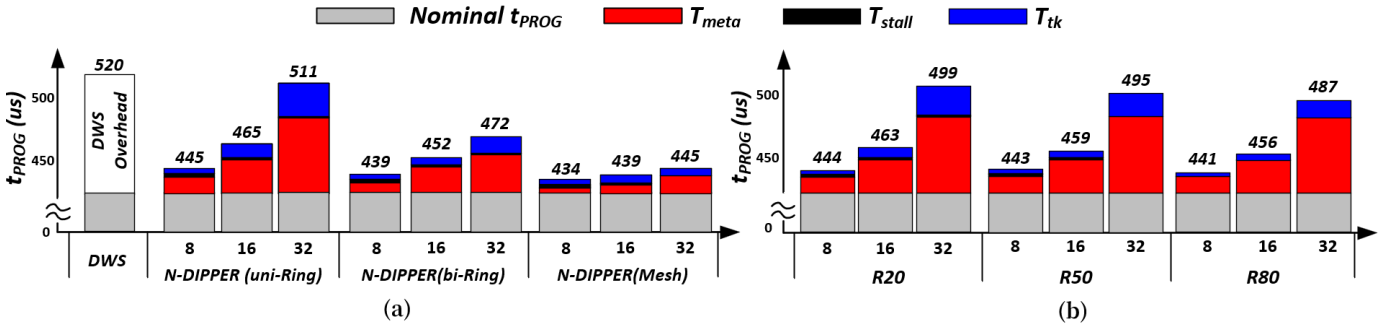


Fig. 12: Program latency (t_{PROG}) for (a) Baseline, DWS, and N–DIPPER (uni-ring/bi-ring/mesh) across die counts for write-only sequential workload and (b) N–DIPPER under different read–write mixes with unidirectional ring topology. T_{meta}: metadata propagation latency, T_{tk}: token waiting time, T_{stall}: stall time.

where the y-axis shows the total peak current and the x-axis is time. Fig. 11(a) shows the baseline without any peak-current management and results in the best performance (in terms of execution time shown on the x-axis). However, since no peak current management is performed, high current phases overlap can occur and the peak current exceeds the package-level budget. [6] In comparison, SCT, DWS, and N–DIPPER (Fig. 11(b–d)) all eliminate peak-current violations but have different impact on performance. SCT is the most conservative, degrading performance (or completion time) by approximately 23% as the SSD controller limits parallelism but does not require any changes to the NAND devices. DWS reduces peak magnitude by lowering the internal charge-pump slew rate – thus, results in performance degradation of approximately 20%. In comparison, N–DIPPER has the smallest performance degradation (approximately 4%) as the high current (HC) phases are scheduled to avoid exceeding the peak current limit but does not require reducing the voltage slope of the wordline ramp or unnecessary throttling. While the results are not shown, when `PPOT[]` (or peak current offset) is not used in N–DIPPER, the performance slowdown increases to approximately 7%. Thus, the device-aware scheduling of N–DIPPER allows it to maximize utilization of the package power budget without worst-case guardbands of SCT or the global throttling of DWS.

C. Program Latency (t_{PROG}) Overhead

To better understand the overhead of peak-current management, especially as the number of dies increases, analysis of a t_{PROG} is shown in Fig. 12, where t_{PROG} is broken down into the nominal t_{PROG} value and the overhead. For N–DIPPER, the overhead is divided into T_{tk} (the average latency to grab or wait for the token), T_{stall} (the amount of time a die stalls when it can potentially exceed the package peak-current budget), and T_{meta} (the amount of time it takes to broadcast the metadata across all dies.) Results for `SCT` are not shown since `SCT` does not impact t_{PROG} but only impacts the scheduling of when the NAND operations occur. `DWS` is shown as a single bar since it uses a fixed waveform-shaping profile to match the PMIC budget and does not vary with die count or NAND access pattern – resulting in a fixed per-die t_{PROG} with a fixed overhead.

Fig. 12(a) shows t_{PROG} overhead as we scale the number of dies per package. Unlike DWS, N–DIPPER overhead increases with die count, and the growth is dominated by two factors: (i) metadata latency (T_{meta}) that is impacted by topology hop count and (ii) token and stall latency (T_{tk} and T_{stall}) when token forwarding and the current-budget check blocks HC entry. N–DIPPER (uni-ring/bi-ring) becomes increasingly sensitive to die count because T_{meta} grows with hop distance. As T_{meta} increases, token forwarding is delayed while metadata is in-flight, which in turn increases T_{tk} by extending the time to acquire the token. A mesh topology effectively reduces the hop count and thus lowers T_{meta} and improves scaling as die count increases. Fig. 12(b) evaluates N–DIPPER under different read–write mixes. As the read ratio increases (R20 $\rightarrow$ R50 $\rightarrow$ R80), the overhead decreases primarily because T_{tk} shrinks: read operations induce smaller and shorter current spikes than program HC phases, leaving more current headroom and reducing budget-induced stalls. In contrast, T_{meta} is largely insensitive to workload mix because it is determined by the topology.

D. Impact of Multi-Token on Scalability

As the number of dies increases, t_{PROG} scheduling overhead of N–DIPPER can increase as T_{meta} and T_{tk} are proportional to the number of dies whereas T_{stall} can be smaller at higher die counts because longer T_{meta} naturally spaces out HC entries, so some earlier HC phases expire before the next admission check. Evaluation of single and multi-tokens on N–DIPPER for different topologies are shown in Figure 13. For multi-token evaluation, an N-die package is partitioned into K token domains (e.g., K=2 for 16 dies, K=4 for 32 dies).

For 8 dies, the improvement from multiple tokens is negligible, since the scheduling overhead is small; however, the benefit increases for larger number of dies. For example, in a 32-die unidirectional Ring, t_{PROG} decreases from 511 µs (single token) to 446 µs, reducing t_{PROG} by 15%. For a Mesh,

[6] As discussed earlier, such violations can degrade performance but the impact of voltage droop, charge-pump stability, etc. is not modeled.

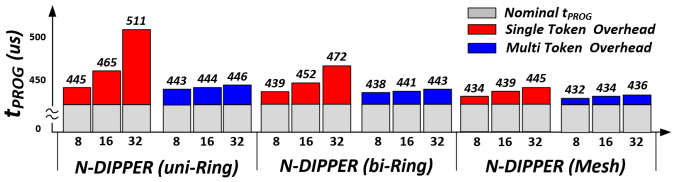
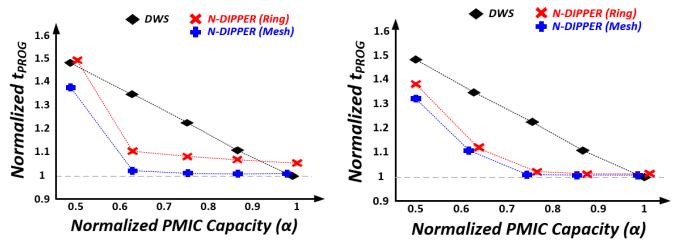


Fig. 13: Comparison of t_{PROG} with single and multi-token for N–DIPPER (8,16, 32 dies) with Ring and Mesh topologies.



(a) 16 dies (b) 32 dies

Fig. 14: Impact of PMIC provision factor (α) on t_{PROG} for (a) 16 dies with single token and (b) 32 dies with multi-token. t_{PROG} is normalized to Baseline for each α.

multi-token reduces t_{PROG} by 2–9 µs. One interesting observation is that with multi-token N–DIPPER, the performance gap between the different topologies can be reduced – e.g., from 15% with a single token to 3% with multi-token for 32 dies.

E. Impact of PMIC Provision Factor

The performance impact of peak current management can depend on the amount of peak current provisioned across all of the dies within a single package by the PMIC. In particular, the PMIC provision factor (α) can be defined as follows,

$$\alpha = \frac{I_{\mathrm{PKG}}}{N \cdot I_{\mathrm{peak,max}}}$$

where N is the number of dies in a package and $I_{\mathrm{peak,max}}$ is the per-die peak current of the dominant HC phase (HC2, 200 mA in our setup). $\alpha = 1$ corresponds to a NAND system where sufficient current is provided to each NAND die and no peak-current management is necessary while when $\alpha < 1$, I_{PKG} does not provide sufficient amount of current and peak current management is necessary to avoid any peak-current violations. In this work, we assumed $\alpha = 0.75$ but we vary α to understand the overhead impact of peak-current management and Fig. 14 plots the normalized t_{PROG} of DWS and N–DIPPER (Ring/Mesh) for 16-dies package (with a single token for N–DIPPER) and 32-dies package with a multi-token configuration. The average t_{PROG} shown is normalized to the Baseline under the same PMIC capacity. For different values α, N–DIPPER results in lower normalized t_{PROG} than DWS for both Ring and Mesh topologies, with the largest benefit under tighter provisioning (smaller α).

For tight PMIC budgets (i.e., low α), DWS aggressively clips HC peaks and thus, increasing t_{PROG} by up to 40–50%. In comparison, N–DIPPER preserves the original HC waveforms and the performance increase comes from the scheduling overhead (i.e., T_{meta}, T_{stall} and T_{tk}). As α increases, the amount of clipping required by DWS is reduced and thus, t_{PROG} is also reduced and for $\alpha = 1$, the overhead from DWS does not exist. However, for N–DIPPER, the scheduling overhead still exists even when $\alpha = 1$ and N–DIPPER results in higher t_{PROG} by 5% compared to DWS. Thus, depending on the actual value of $\alpha = 1$, there can be a trade-off across the different peak current managements schemes (as well as the topology for N–DIPPER); however, for reasonable values of α (i.e., $0.6 < \alpha < 0.9$), N–DIPPER outperforms DWS – with the mesh implementation of N–DIPPER providing lower t_{PROG} compared to a ring implementation (but at the cost of higher overhead in-package routing and additional Tx/Rx ports).

F. End-to-End Performance Evaluation

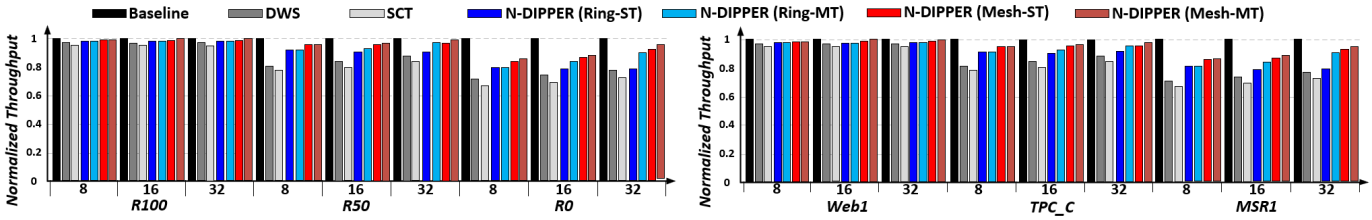
The impact of peak-current management on end-to-end performance is shown in Fig. 15 where throughput or I/O operations per second (IOPS) normalized to the Baseline is shown, for both synthetic workload and real traces [44], [46]. Overall, N–DIPPER provides throughput that is similar to Baseline across all of the workloads (while ensuring peak-current violations do not occur). For read-intensive workloads (e.g., R50/R100 for synthetic traces and `Web1`, TPC-C from real traces), the impact of peak current management is relatively small, as read operations consume significantly lower peak current than program operations. Consequently, N–DIPPER achieves throughput that is within 3% of the Baseline. In comparison, even for read-intensive workloads, SCT and DWS result in up to 22% (19%) throughput degradation for the `R50` synthetic workload. Performance degradation is higher as the amount of write traffic increases. For write-intensive workloads (e.g., R0 for synthetic traces and `MSR1` from real traces), the performance degradation for N–DIPPER can be up to 18%, compared to the baseline. However, multiple token and mesh topology can help achieve higher performance, allowing N–DIPPER to exceed the throughput of SCT (DWS) by up to 16% (22%), respectively. The relative degradation diminishes at higher die counts because the system transitions into a channel-bound regime, where the saturation of the shared bus effectively masks the additional latencies introduced by SCT and DWS.

VI. Discussion

In this section, we discuss some of the limitations of N–DIPPER as well as future work to explore inter-die network. The cost/overhead of N–DIPPER is also analyzed and the related work is also presented.

A. Limitations

In this work, we focus on *in-package* scheduling among dies that share a common rail to prevent package-level PMIC-limit violations while maintaining high performance (or throughput). However, from an overall SSD system perspective, *inter*-package peak-current management must be properly managed



(a) (b)

Fig. 15: System throughput normalized to Baseline with no peak-current management for (a) synthetic traces with read–write mixes (R100/R50/R0) and (b) real-world traces from SNIA IOTTA, as the number of dies is varied (8,16,32).

TABLE IV: Hardware overhead for N–DIPPER controller.

Item	Type	Entries	Bits	Area (mm^2)
I_p, t_{HC}	eFuse	10	8	0.0086
PPOT	eFuse	135	8	0.1161
PCSR	Register	64	16	0.6000
TX /RX	PAD	8	–	0.0192
Controller logic	Gates	≈1K	–	0.0017
	Total area overhead			**0.7456**

to ensure that overall system peak current violation does not occur. In this work, we assumed a *hierarchical* peak current management where the amount of peak current allocated to each package is statically determined or partitioned but it remains to be seen how efficient peak-current management can be achieved across different packages and/or whether N–DIPPER can be potentially extended to inter-package peak-current management.

Another challenge of hierarchical peak-current management (as well as the multi-token approach proposed in this work) is load imbalance from static partitioning. For example, since multi-token N–DIPPER partitions an N-die package into K independent token domains and enforces a per-domain budget of I_{pkg}/K, each domain can utilize only its locally allocated current budget. As a result, if workload demand is skewed (e.g., write/program traffic concentrates in one domain while other domains are idle or read-dominant), one domain might suffer from unnecessary stalls (or performance degradation) while the other domain's current budget goes underutilized. In comparison, single-token N–DIPPER maintains a unified package-wide view and can exploit the full I_{pkg} under such imbalance, potentially avoiding any unnecessary stalls and achieving higher throughput. Addressing this limitation likely requires *dynamic* peak-current budget reallocation across domains.

In addition, N–DIPPER uses a single token that serves two roles: granting HC-entry as well as allowing the node (or die) to broadcast its metadata information. An alternative design could decouple these roles by using separate tokens – allow overlap of these two tokens across the package and potentially reduce T_{tk} and/or T_{meta}.

B. Hardware Overhead

The hardware overhead of N–DIPPER is dominated by the additional on-chip storage, as summarized in Table IV. To estimate the storage and logic area overhead, we implemented and synthesized the N–DIPPER controller logic and storage components (eFuses and registers) using Synopsys Design Compiler with a 45nm standard-cell library [47]. For the additional Tx/Rx ports, we conservatively assume a pad area of ≈2,400 μm^2 per port (40 μm pad pitch × 60 μm peripheral height) [28] and for the worst-case, 8 ports are needed when using the mesh topology. The area introduced by the additional storage is approximately 0.7247 mm^2, while the additional I/O (TX/RX) add approximately 0.0192 mm^2, and the controller logic itself is approximately 0.0017 mm^2. Although modern 3D NAND peripheral circuits (e.g., Samsung V8) likely utilize a finer process node (∼20 nm class) [48] compared to our 45 nm library, we directly compare our synthesized area against the total die area of 88.6 mm^2 [2] to provide a conservative upper bound. Thus, even under this conservative estimate, the area overhead from N–DIPPER is approximately only **0.84%**.

C. Related Work

Peak Current Management: As peak currents becomes more of a critical constraint in NAND flash systems with increased number of word line layers and die-level parallelism, different peak-current management schemes have been proposed and used on modern SSD systems. Some SSD controllers implement conservative scheduling to limit peak current by restricting the number of concurrent operations across dies or planes [49]–[51]. While such approach does not require any modification to the NAND devices, the inter-die timing drift can not be estimated (or known) by the SSD controller and results in performance trade-off by enforcing worst-case margins or guardbands. In addition, controller-based scheduling can limit scalability as the number of dies increases. Device-level management [8] has been proposed where instantaneous peak current magnitude is reduced by shaping wordline voltage ramps [17], [19]; however, this comes at the cost of increasing the setup time for every program operations – thus, degrading the write throughput [8]. Modern NAND also employ hardware safety circuits (e.g., Low-Voltage Detection (LVD) or brownout protection) that detect V_{CC} droops and clock-gate

or stall NAND operations; however, such mechanisms are *reactive* protection and introduce significant latency penalties due to operation aborts or retries. In comparison, N–DIPPER is a distributed peak current management that exploits communication between the NAND dies to *schedule* peak currents or high-current phases.

SSD Architectures: Interconnection networks have been proposed within SSD systems, including within the SSD [52] as well between flash memory chips [53], [54], to maximize parallelism and performance. This work also proposes interconnection networks in a SSD system but it is leveraged within a single package (between the dies) for peak-current management. There has been a significant amount of work on different aspects of SSD architectures, including performance [49], [53], [55], [56], near-flash computing [57], [58], power [7], [37], [59], and reliability [33], [60]. However, to the best of our knowledge, this is one of the first works to explore architectural changes to enable efficient peak current management in NAND-based systems. The concept of *tokens* has been widely used in computer systems as an abstraction to manage resources or regulate access, often in a distributed system, to avoid global synchronization while providing fairness and/or minimizing congestion [38]. Some examples include traditional credit-based flow control [38], token-based flow controls [61], end-to-end flow control [62], and token coherence [63]. Tokens have also been proposed for NAND power management through Multi-Token based Power Management (MTPM) [21] where each token represents a pre-defined current amount and multiple tokens need to be obtained to enter high-current phase. However, MTPM is an *on-demand* approach that can only provide coarse-grain estimate of peak-currents and is not necessarily scalable; in comparison, N–DIPPER presented a fine-grain approach that exploits device-aware information to enable a more accurate estimate of peak current and enable a more scalable peak-current management.

VII. Conclusion

In this work, we presented N–DIPPER, an inter-die peak power management architecture for high-density 3D NAND packages that exploits an inter-die fabric for peak-current management. With token-based scheduling, through an in-package network, N–DIPPER schedules high-current (HC) phases of NAND operations to avoid peak-current violations while minimizing any impact on performance. Our evaluations show that N–DIPPER can achieve throughput that is within 3% of the baseline system without any peak-current management while guaranteeing that peak current violations do not occur.

Acknowledgments

We would like to thank the anonymous reviewers for their comments. This work is supported in part by IITP grants funded by MSIT (no. RS-2023-00228255, RS-2024-00402898, RS-2025-02214654, RS-2025-02217106), and in part by NRF (no. RS-2023-NR077034).

References

[1] M. Sako, Y. Watanabe, T. Nakajima, J. Sato, K. Muraoka, M. Fujiu, F. Kono, M. Nakagawa, M. Masuda, K. Kato, Y. Terada, Y. Shimizu, M. Honma, A. Imamoto, T. Araya, H. Konno, T. Okanaga, T. Fujimura, X. Wang, M. Muramoto, M. Kamoshida, M. Kohno, Y. Suzuki, T. Hashiguchi, T. Kobayashi, M. Yamaoka, and R. Yamashita, "A low power 64 gb mlc nand-flash memory in 15 nm cmos technology," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 1, pp. 196–203, 2016.

[2] M. Kim, S. W. Yun, J. Park, H. K. Park, J. Lee, Y. S. Kim, D. Na, S. Choi, Y. Song, J. Lee, K. Kyung, J. Park, J. Kim, M. Ahn, S. Kanasaki, H. Jang, Y. Park, W. Cho, C. Lee, S. Shin, S. Park, K. Roh, H.-S. Oh, S.-C. Jeon, D. Jang, J.-Y. Lee, P. S. Kwak, and S.-W. Nam, "A 1tb 3b/cell 8th-generation 3d-nand flash memory with 164mb/s write throughput and a 2.4 gb/s interface," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 136–137.

[3] S.-S. Park, J.-D. Lyu, M. Kim, J. Lee, Y. Song, C.-H. Yu, H. Makoto, Y. Kwon, J.-H. Park, H.-J. Kim, D. Lee, D. Seo, B. Go, S. Jeon, Y. Kim, D.-H. Kim, Y. Jo, H. Yoon, J. Park, I. Kim, S. Kim, H. Lee, J.-H. Yu, S.-L. Kim, H.-S. Ku, J. Seo, J. Byun, S.-H. Yun, K. Kang, S.-B. Kim, Y. Lee, Y. Lee, K. Kang, H.-J. Lee, Y. Ryu, H. Kim, W. Kim, H. Choi, J. Jeon, A. Park, R. Song, J.-H. Kim, J.-S. Kim, H.-S. Lee, M.-K. Lee, J.-I. Son, J. Cho, M. Kim, J.-W. Im, J. Park, H. Kwon, Y. Choi, C. Yoon, S. Lee, K. Song, and S.-H. Hur, "A 28Gb/mm^2 4XX-layer 1Tb 3b/cell WF-bonding 3D-NAND flash with 5.6Gb/s/pin IOs," in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, Feb. 2025, pp. 424–426.

[4] J. Li, P. Liu, N. Li, J. Wang, G. Zhou, J. Zhai, and H. Qian, "Practically scalable transmission of large models for deep learning training," in *USENIX Conference on File and Storage Technologies (FAST)*, 2022, pp. 1–16.

[5] J. Kim and S. Cho, "3-d nand flash memory scaling: Status, challenges, and future perspectives," *IEEE Transactions on Electron Devices*, vol. 68, no. 11, pp. 5466–5472, 2021.

[6] F. Chen, B. Hou, and R. Lee, "Internal parallelism of flash memory-based solid-state drives," *ACM Transactions on Storage (TOS)*, vol. 12, no. 3, pp. 1–39, 2016.

[7] T. Hwang, B. Kim, and M. Jung, "Thermal throttling of ssds: The forgotten performance killer," in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1–13.

[8] R. Micheloni, L. Crippa, and A. Marelli, *Inside NAND Flash Memories*. Springer Science & Business Media, 2010.

[9] C. Huang, F. Liu, Q. Wang, and Z. Huo, "Low power program scheme with capacitance-less charge recycling for 3d nand flash memory," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 68, no. 7, pp. 2478–2482, 2021.

[10] K.-S. Yoon, H.-S. Kim, W. Qu, Y.-S. Yuk, and G.-H. Cho, "Fully integrated digitally assisted low-dropout regulator for a nand flash memory system," *IEEE Transactions on Power Electronics*, vol. 33, no. 1, pp. 388–406, 2018.

[11] N. Sudo, "Power down detection circuit and semiconductor memory device," U.S. Patent Application US20220005511A1, Jan. 2022, assignee: Winbond Electronics Corp., published Jan. 6, 2022. [Online]. Available: https://patents.google.com/patent/US20220005511A1/en

[12] SK hynix, "One-team spirit: Sk hynix's 321-layer nand," https://news.skhynix.com/one-team-spirit-the-miracle-of-321-layers/, 2024, accessed: August 01, 2025.

[13] C. Siau, K.-H. Kim, S. Lee, K. Isobe, N. Shibata, K. Verma, T. Ariki, J. Li, J. Yuh, A. Amarnath, Q. Nguyen, O. Kwon, S. Jeong, H. Li, H.-L. Hsu, T.-y. Tseng, S. Choi, S. Darne, P. Anantula, A. Yap, H. Chibvongodze, H. Miwa, M. Yamashita, M. Watanabe, K. Hayashi, Y. Kato, T. Miwa, J. Y. Kang, M. Okumura, N. Ookuma, M. Balaga, V. Ramachandra, A. Matsuda, S. Kulkani, R. Rachineni, P. K. Manjunath, M. Takehara, A. Pai, S. Rajendra, T. Hisada, R. Fukuda, N. Tokiwa, K. Kawaguchi, M. Yamaoka, H. Komai, T. Minamoto, M. Unno, S. Ozawa, H. Nakamura, T. Hishida, Y. Kajitani, and L. Lin, "13.5 a 512gb 3-bit/cell 3d flash memory on 128-wordline-layer with 132mb/s write performance featuring circuit-under-array technology," in *2019 IEEE International Solid-State Circuits Conference - (ISSCC)*, 2019, pp. 218–220.

[14] J. Yuh, J. Li, H. Li, Y. Oyama, C. Hsu, P. Anantula, S. Jeong, A. Amarnath, S. Darne, S. Bhatia, T. Tang, A. Arya, N. Rastogi, N. Ookuma, H. Mizukoshi, A. Yap, D. Wang, S. Kim, Y. Wu, M. Peng, J. Lu, T. Ip, S. Malhotra, D. Han, M. Okumura, J. Liu, J. Sohn, H. Chibvongodze, M. Balaga, A. Matsuda, C. Puri, C. Chen, I. K. V, C. G, V. Ramachandra, Y. Kato, R. Kumar, H. Wang, F. Moogat, I.-S. Yoon, K. Kanda, T. Shimizu, N. Shibata, T. Shigeoka, K. Yanagidaira, T. Kodama, R. Fukuda, Y. Hirashima, and M. Abe, "A 1-tb 4b/cell 4-plane 162-layer 3d flash memory with a 2.4-gb/s i/o speed interface," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 130–132.

[15] J.-W. Park, D. Kim, S. Ok, J. Park, T. Kwon, H. Lee, S. Lim, S.-Y. Jung, H. Choi, T. Kang, G. Park, C.-W. Yang, J.-G. Choi, G. Ko, J. Shin, I. Yang, J. Nam, H. Sohn, S.-I. Hong, Y. Jeong, S.-W. Choi, C. Choi, H.-S. Shin, J. Lim, D. Youn, S. Nam, J. Lee, M. Ahn, H. Lee, S. Lee, J. Park, K. Gwon, W. Jeong, J. Choi, J. Kim, and K.-W. Jin, "A 176-stacked 512gb 3b/cell 3d-nand flash with 10.8gb/mm² density with a peripheral circuit under cell array architecture," in *IEEE International Solid-State Circuits Conference (ISSCC)*. San Francisco, CA, USA: IEEE, Feb 2021, pp. 422–423.

[16] W. Jung, H. Kim, D.-B. Kim, T.-H. Kim, N. Lee, D. Shin, M. Kim, Y. Rho, H.-J. Lee, Y. Hyun, J. Park, T. Kim, H. Kim, G. Lee, J. Lee, J. Jang, J. Park, S. Kim, S. C. Jeon, S. Kim, J.-H. Song, M.-S. Kim, T. Lee, B.-K. Chun, T. Kim, Y. G. Lee, H. Lee, S. Lee, H. Lee, D. Cho, S.-W. Nam, Y. Kim, K. Yoon, Y. Lee, S. Kim, J. Hwang, R. Song, H. Jang, J. Son, H. Jeon, M. Lee, M. Lee, K. Kim, E. Lee, M. Lee, S. Jo, C. H. Kim, J. C. Park, K. Yun, S. Seol, J.-H. Cho, S. Lee, J.-Y. Lee, and S.-H. Hur, "13.3 a 280-layer 1tb 4b/cell 3d-nand flash memory with a 28.5gb/mm2 areal density and a 3.2gb/s high-speed io rate," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, 2024, pp. 236–237.

[17] S.-K. Won, Y. Noh, H. Cho, J. Ryu, S. Choi, S. Choi, D. Kim, J. Chung, B. Han, and E.-Y. Chung, "High-voltage wordline generator for low-power program operation in nand flash memories," in *IEEE Asian Solid-State Circuits Conference 2011*, 2011, pp. 169–172.

[18] B.-S. You, J.-S. Park, S.-D. Lee, G.-H. Baek, J.-H. Lee, M.-S. Kim, J.-W. Kim, H. Chung, E.-S. Jang, and T.-Y. Kim, "A high performance co-design of 26 nm 64 gb mlc nand flash memory using the dedicated nand flash controller," *JSTS: Journal of Semiconductor Technology and Science*, vol. 11, no. 2, pp. 121–129, 2011.

[19] M. R. Garin, R. E. Scheuerlein, M. Q. Tran, and R. W. Van Buskirk, "Non-volatile memory and method with fast setup of word line voltage," US Patent US8 605 551B2, Dec. 10, 2013, filed May 14, 2007. [Online]. Available: https://patents.google.com/patent/US8605551B2

[20] L.-P. Chang, C.-H. Cheng, S.-T. Chang, and P.-H. Chou, "Current-aware flash scheduling for current capping in solid state disks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 2, pp. 321–334, 2020.

[21] T. You, S. Han, Y. M. Park, H.-J. Lee, and E.-Y. Chung, "Multitoken-based power management for nand flash storage devices," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2898–2910, 2020.

[22] J. Park, S. Yoo, S. Lee, and C. Park, "Power modeling of solid state disk for dynamic power management policy design in embedded systems," in *IFIP International Workshop on Software Technolgies for Embedded and Ubiquitous Systems*. Springer, 2009, pp. 24–35.

[23] C. Zambelli, R. Micheloni, L. Crippa, L. Zuolo, and P. Olivo, "Impact of the nand flash power supply on solid state drives reliability and performance," *IEEE Transactions on Device and Materials Reliability*, vol. 18, no. 2, pp. 247–255, 2018.

[24] R.-C. Yang, K.-C. Lai, C.-F. Wu, C.-C. Lin, W.-T. Lin, and C.-Y. Lin, "Techniques for the fast settling of word line voltages in non-volatile memory devices," US Patent US8 374 031B2, Feb. 12, 2013, filed May 13, 2011. [Online]. Available: https://patents.google.com/patent/US8374031B2

[25] K.-D. Suh, B.-H. Suh, Y.-H. Lim, J.-K. Kim, Y.-J. Choi, Y.-N. Koh, S.-S. Lee, S.-C. Kwon, B.-S. Choi, J.-S. Yum, J.-H. Choi, J.-R. Kim, and H.-K. Lim, "A 3.3 v 32 mb nand flash memory with incremental step pulse programming scheme," *IEEE Journal of Solid-State Circuits*, vol. 30, no. 11, pp. 1149–1156, 1995.

[26] J. Sheng, P. Xu, M. Qiu, Y. Luo, L. Ding, Z. Yao, B. He, S. Gou, and W. Ma, "Bit upset and performance degradation of nand flash memory induced by total ionizing dose effects," *AIP Advances*, vol. 13, no. 4, 2023.

[27] M. M. Navidi and D. W. Graham, "A regulated charge pump with extremely low output ripple," *Electronics*, vol. 8, no. 11, p. 1293, 2019.

[28] R. Micheloni, A. Marelli, and K. Eshghi, *3D Flash Memories*. Springer, 2016, chapter 2: 3D NAND Architectures and Packaging.

[29] J. K. Park and S. E. Kim, "A review of cell operation algorithm for 3d nand flash memory," *Applied Sciences*, vol. 12, no. 21, p. 10697, 2022.

[30] Y. Cai, E. F. Haratsch, O. Mutlu, and K. Mai, "Error patterns in MLC NAND flash memory: Measurement, characterization, and analysis," in *Proceedings of the Design, Automation & Test in Europe Conference (DATE)*, 2012, pp. 1–6.

[31] Y. Cai, S. Ghose, Y. Luo, K. Mai, and O. Mutlu, "Vulnerabilities in 3D NAND flash memories: Communication, protection, and recovery," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2017, pp. 349–360.

[32] D. w. Kwon, D.-B. Kim, J. Lee, S. Kim, R. Lee, J.-H. Lee, and B.-G. Park, "Analysis on new read disturbance induced by hot carrier injections in 3-d channel-stacked nand flash memory," *IEEE Transactions on Electron Devices*, vol. 66, no. 8, pp. 3326–3330, 2019.

[33] B. Schroeder, R. Lagisetty, and A. Merchant, "Flash reliability in production: The expected and the unexpected," in *USENIX Conference on File and Storage Technologies (FAST)*, 2016, pp. 157–170.

[34] A. Prodromakis, S. Korkotsides, and T. Antonakopoulos, "Mlc nand flash memory: Aging effect and chip/channel emulation," *Microprocessors and Microsystems*, vol. 39, no. 8, pp. 1052–1062, 2015.

[35] S. Bou-Sleiman and M. Ismail, "Dynamic self-regulated charge pump with improved immunity to pvt variations," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 22, no. 8, pp. 1716–1729, 2014.

[36] X. Ma, L. Wang, and J. Yu, "A 5 V-to-32 V input PVT-robust charge-pump circuit with adjustable output in a 0.18 μm BCD process," *Electronics*, vol. 11, no. 18, p. 2828, 2022.

[37] V. Mohan, S. Gurumurthi, and M. R. Stan, "FlashPower: A detailed power model for NAND flash memory," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2013, pp. 502–507.

[38] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. Morgan Kaufmann, 2004.

[39] J. Wang, F. Duan, Z. Lv, S. Chen, X. Yang, H. Chen, and J. Liu, "A short review of through-silicon via (tsv) interconnects: metrology and analysis," *Applied Sciences*, vol. 13, no. 14, p. 8301, 2023.

[40] H. Ken, W.-S. Choi, Y.-H. Kim, H.-S. Oh, I.-S. Park, and Y.-H. Jun, "A 1.3 μw/0.8°C-inaccuracy on-die temperature sensor with time-domain readout for mobile DRAM and NAND flash memory," *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 53, no. 11, pp. 3106–3116, 2018.

[41] H. Lee, W. Park, D. Youn, and S. Kim, "Non-volatile memory device having temperature sensor and method of operating the same," Nov 2018, uS Patent 10,127,982.

[42] R. Dutt, N. N. Yang, and C. Avila, "Start voltage adjustment for NAND flash memory programming," Feb 2014, uS Patent 8,644,073.

[43] M. Jung, J. Zhang, A. Abulila, M. Kwon, N. Shahidi, J. Shalf, N. S. Kim, and M. Kandemir, "Simplessd: Modeling solid state drives for holistic system simulation," *IEEE Computer Architecture Letters*, vol. 17, no. 1, pp. 37–41, 2018.

[44] Storage Networking Industry Association, "Iotta repository: Input/output traces, tools, and analysis," https://iotta.snia.org/, 2024, accessed: August 01, 2025.

[45] L.-P. Chang, C.-H. Cheng, and K.-H. Lin, "A flash scheduling strategy for current capping in multi-power-mode ssds," in *2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2017, pp. 566–571.

[46] D. Narayanan, A. Donnelly, and A. Rowstron, "Write off-loading: Practical power management for enterprise storage," in *USENIX Conference on File and Storage Technologies (FAST)*, 2008, pp. 253–267.

[47] Nangate Inc., "Nangate 45nm Open Cell Library," https://si2.org/open-cell-library/, 2008, accessed: 2026-01-15.

[48] J. Choe, "NAND Flash Memory Technology Roadmap and Technology Analysis," in *Proceedings of Flash Memory Summit (FMS)*, Santa Clara, CA, Aug. 2023, techInsights analysis contrasting Samsung's COP peripheral process nodes (∼20nm) with hybrid-bonding based advanced logic nodes.

[49] N. Agrawal, V. Ganapathy, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Design tradeoffs for SSD performance," in *USENIX Annual Technical Conference (ATC)*, 2008, pp. 57–70.

[50] M. Jung and M. Kandemir, "Sprinkler: Maximizing resource utilization in many-chip solid state disks," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2014.

[51] F. Chen, R. Zhang, and X. Zhang, "Essential roles of exploiting internal parallelism of flash memory on ssd performance in high-speed i/o interface," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.

[52] J. Kim, M. Jung, and J. Kim, "Decoupled SSD: Rethinking SSD Architecture through Network-based Flash Controllers," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–15.

[53] R. Nadig, M. Sadrosadati, H. Mao, N. Mansouri-Ghiasi, A. Tavakkol, J. Park, H. Sarbazi-Azad, J. Gómez-Luna, and O. Mutlu, "Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 36:1–36:16.

[54] J. Kim, S. Kang, Y. Park, and J. Kim, "Networked SSD: Flash Memory Interconnection Network for High-Bandwidth SSD," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 388–403.

[55] A. Gupta, Y. Kim, and B. Urgaonkar, "Dftl: a flash translation layer employing demand-based selective caching of page-level address mappings," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009, pp. 229–240.

[56] A. Tavakkol, J. Gomez-Luna, M. Sadrosadati, and O. Ghose, Saugata andMutlu, "MQSim: A framework for enabling realistic studies of modern multi-queue SSD devices," in *USENIX Conference on File and Storage Technologies (FAST)*, 2018, pp. 49–66.

[57] S. Seshadri, M. Gahagan, S. Bhaskaran, T. Bunker, A. De, Y. Jin, Y. Liu, and S. Swanson, "Willow: A user-programmable SSD," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2014, pp. 67–80.

[58] B. Gu, A. S. Yoon, D.-H. Bae, I. Jo, J. Lee, J. Yoon, J.-U. Kang, M. Kwon, C. Yoon, S. Cho, J. Jeong, and D. Kim, "Biscuit: A framework for near-data processing of big data workloads on solid state drives."

[59] B. Yoo, Y. Won, S. Cho, S. Kang, J. Choi, and S. Yoon, "Ssd characterization: From energy consumption's perspective," in *3rd Workshop on Hot Topics in Storage and File Systems (HotStorage 11)*, 2011.

[60] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "A large-scale study of flash memory failures in the field," in *ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, 2015, pp. 177–190.

[61] A. Kumar, L.-S. Peh, and N. K. Jha, "Token flow control," in *Proceedings of the 41st IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2008, pp. 342–353.

[62] H. Kim, Y. Kim, and J. Kim, "Clumsy flow control for high-throughput bufferless on-chip networks," *IEEE Computer Architecture Letters*, vol. 12, no. 2, pp. 47–50, 2013.

[63] M. M. K. Martin, M. D. Hill, and D. A. Wood, "Token coherence: decoupling performance and correctness," in *Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA)*, 2003, pp. 182–193.